ASSIGNMENT 1

PAYMENT CALCULATOR FOR A VILLAGE MILK CO-OPERATIVE

Task 4: Write Correct Expressions

Aim: To calculate fat steps, SNF steps, milk rate and payment amount using correct Python expressions.

Expressions:

Fat steps = `round((fat - BASE_FAT) / FAT_STEP)`

SNF steps = `round((snf - BASE_SNF) / SNF_STEP)`

Rate = `BASE_RATE + fat_steps * FAT_ADJUSTMENT + snf_steps * SNF_ADJUSTMENT`

Amount = `litres * rate`

Operator precedence: Python evaluates parentheses first, followed by exponentiation, unary operators and multiplication/division before addition/subtraction. Parentheses make the intended calculation clear.

Floating-point surprise: `(4.2 - 3.5) / 0.1` may produce `7.000000000000001` because binary floating-point cannot represent some decimal values exactly. Using `round()` gives the intended step count of 7 for this example.

Comparison chaining: `2.0 <= fat <= 7.0` checks whether fat is within the permitted range. The Boolean operators `and`, `or` and `not` combine or reverse conditions.

Unexpected results:

`0.1 + 0.2 == 0.3` gives `False` because of floating-point precision.

`int(-3.7)` gives `-3` because `int()` truncates toward zero.

`math.floor(-3.7)` gives `-4` because `floor()` rounds down to the next integer.

Alternative concept: For fat = 4.2, the ideal step count is 7. For fat = 3.1, it is -4. Using `int()` can truncate floating-point results and give incorrect step counts; `math.floor()` can also shift negative values downward. `round()` is suitable for these examples, provided floating-point precision and rounding behaviour are considered.

Complexity: Each calculation uses a fixed number of operations, so time complexity is O(1) and auxiliary space complexity is O(1).

Conclusion: Correct expressions, appropriate parentheses and rounded step counts help calculate milk payments accurately.

